Compulsory Assignment 2: Convolutional neural networks

Please fill out the group name, number, members and optionally the name below.

Group number: 45 **Group member 1**: Mehdi Jamali **Group member 2**: Oliver Tschudi Madsen **Group member 3**: Tobias Tessem Hole **Group name (optional)**: MOT Squad

Assignment Submission

To complete this assignment answer the relevant questions in this notebook and write the code required to implement the relevant models. The assignment is submitted by handing in this notebook as an .ipynb file and as a .pdf file.

Introduction

In this assignment, you will build a Convolutional Neural Network (CNN) to classify images of natural scenes from around the world. Dataset: Intel-image-classification https://www.kaggle.com/datasets/puneet6060/intel-image-classification

This Data contains around 25k images of size 150x150 distributed under 6 categories.
{'buildings' -> 0, 'forest' -> 1, 'glacier' -> 2, 'mountain' -> 3, 'sea' -> 4, 'street' -> 5 }

In CA2, we use only 3 classes "buildings", "forest", "sea".

This data was initially published on https://datahack.analyticsvidhya.com by Intel to host a Image classification Challenge.

Landscape Pictures

Example image:

Assignment structure

1. Part 0: Setup & Data
2. Part 1: Baseline CNN (Clean Data)
3. Part 2: Choose Your Robustness Challenges
4. Part 3: Results & Comparison

```
```

Library imports

```
# Feel free to add or remove libraries as you want
import os
import time
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import sklearn as sk
import tensorflow as tf
import tensorflow.keras as ks
```

Part 0: Setup & Data, CNN for Landscape Picture dataset

NOTE FOR STUDENTS:

Do not forget to change the path of root_dir below so it points to the Intel folder which contains the dataset inside your CA2 directory.

Example:

Windows: r"C:\Users\yourname...\CA2\Intel"

Mac/Linux: "/home/yourname/CA2/Intel"

Loading DATASET

```
import importlib
import utilities

# Force reload the updated utilities.py
importlib.reload(utilities)

from utilities import load_intel_dataset


# Load dataset
data = load_intel_dataset(
    root_dir=r"C:\Users\tobia\OneDrive - Norwegian University of Life
Sciences\skole\dat300\CA2\Intel",
    img_size=(224, 224),
    selected_classes=["buildings", "forest", "sea"],  #  only 3
classes
    verbose=1
```

```
)

X_train, y_train = data["X_train"], data["y_train"]
X_val,   y_val   = data["X_val"],   data["y_val"]
X_test,  y_test  = data["X_test"],  data["y_test"]
class_names      = data["class_names"]

print("Train:", X_train.shape, y_train.shape)
print("Val:",   X_val.shape,   y_val.shape)
print("Test:",  X_test.shape,  y_test.shape)

# Show number of images per class
def count_per_class(y, split_name):
    unique, counts = np.unique(y, return_counts=True)
    print(f"\n{split_name} set class distribution:")
    for cls, cnt in zip(unique, counts):
        print(f"  {class_names[cls]}: {cnt}")

count_per_class(y_train, "Train")
count_per_class(y_val,   "Val")
count_per_class(y_test,  "Test")
```

```
[train] Loaded 500/6736 images
[train] Loaded 1000/6736 images
[train] Loaded 1500/6736 images
[train] Loaded 2000/6736 images
[train] Loaded 2500/6736 images
[train] Loaded 3000/6736 images
[train] Loaded 3500/6736 images
[train] Loaded 4000/6736 images
[train] Loaded 4500/6736 images
[train] Loaded 5000/6736 images
[train] Loaded 5500/6736 images
[train] Loaded 6000/6736 images
[train] Loaded 6500/6736 images
[train] Loaded 6736/6736 images
[test] Loaded 500/1421 images
[test] Loaded 1000/1421 images
[test] Loaded 1421/1421 images
Final shapes → Train: (6062, 224, 224, 3), Val: (674, 224, 224, 3),
Test: (1421, 224, 224, 3)
Classes: ['buildings', 'forest', 'sea']
Train: (6062, 224, 224, 3) (6062,)
Val: (674, 224, 224, 3) (674,)
Test: (1421, 224, 224, 3) (1421,)

Train set class distribution:
  buildings: 1972
  forest: 2044
  sea: 2046
```

```
Val set class distribution:
  buildings: 219
  forest: 227
  sea: 228

Test set class distribution:
  buildings: 437
  forest: 474
  sea: 510
```

Preprocessing

```
# Normalizing input between [0,1]
X_train = X_train.astype("float32") / 255.0 if X_train.size else
X_train
X_val   = X_val.astype("float32")   / 255.0 if X_val.size else X_val
X_test  = X_test.astype("float32")  / 255.0 if X_test.size else X_test

# Converting targets from numbers to categorical format
if y_train is not None and y_train.size:
    num_classes = len(np.unique(y_train))
    y_train = ks.utils.to_categorical(y_train, num_classes)

if y_val is not None and y_val.size:
    y_val = ks.utils.to_categorical(y_val, num_classes)

if y_test is not None and y_test.size:
    y_test = ks.utils.to_categorical(y_test, num_classes)

print("X_train shape:", X_train.shape)
print("X_val shape:",   X_val.shape)
print("X_test shape:",  X_test.shape)


X_train shape: (6062, 224, 224, 3)
X_val shape: (674, 224, 224, 3)
X_test shape: (1421, 224, 224, 3)
```

Part 1: Build a CNN network with the LeNet5 architecture

Implement LeNet5 architecture for Landscape Pictures (RGB):

The LeNet architecture takes a 32×32×C image as input, where C is the number of color channels. You may use resize_dataset() and f1_score() from utilities.py¶

Input & resizing: Resize all images to 32×32 and use C = 3 channels (RGB). If you choose a different input size, update the intermediate shapes accordingly, but keep the LeNet-5 pattern (Conv → Pool → Conv → Pool → FC → FC → FC).

Layer 1 - Convolution (5x5): The output shape should be 28x28x6. **Activation:** ReLU.

MaxPooling: The output shape should be 14x14x6.

Layer 2 - Convolution (5x5): The output shape should be 10x10x16. **Activation:** ReLU.

MaxPooling: The output shape should be 5x5x16.

Flatten: Flatten the output shape of the final pooling layer such that it's 1D instead of 3D. You may need to use tf.reshape.

Layer 3 - Fully Connected: This should have 120 outputs. **Activation:** ReLU.

Layer 4 - Fully Connected: This should have 84 outputs. **Activation:** ReLU.

Layer 5 - Fully Connected (output): `num_classes`. **Activation:** softmax

---

Compile the network with the

- `tf.keras.losses.CategoricalCrossentropy` loss function
- the `adam` optimizer
- with the `accuracy` metric and (your own implementation of the) F1-score metric.

```
# First import tensorflow to make it available
import importlib

# Force reload the updated utilities.py
import utilities
importlib.reload(utilities)
from utilities import resize_dataset

print("Original shapes:")
print("X_train:", X_train.shape)
print("X_val:", X_val.shape)
print("X_test:", X_test.shape)

# Resize to 32x32
X_train = resize_dataset(X_train, target_size=(32, 32))
X_val = resize_dataset(X_val, target_size=(32, 32))
X_test = resize_dataset(X_test, target_size=(32, 32))

print("\nAfter resizing to 32x32:")
print("X_train:", X_train.shape)
print("X_val:", X_val.shape)
print("X_test:", X_test.shape)
```

```
Original shapes:
X_train: (6062, 224, 224, 3)
```

```
X_val: (674, 224, 224, 3)
X_test: (1421, 224, 224, 3)

After resizing to 32x32:
X_train: (6062, 32, 32, 3)
X_val: (674, 32, 32, 3)
X_test: (1421, 32, 32, 3)
```

```
def build_lenet5(input_shape, num_classes):
    model = ks.Sequential()
    model.add(ks.Input(shape=input_shape))
    # Layer 1: Conv -> ReLU -> MaxPool
    model.add(ks.layers.Conv2D(6, kernel_size=(5, 5),
activation='relu'))
    model.add(ks.layers.MaxPooling2D(pool_size=(2, 2)))
    print("\nAfter layer1 and maxpooling:")
    print(model.output_shape)

    # Layer 2: Conv -> ReLU -> MaxPool
    model.add(ks.layers.Conv2D(16, kernel_size=(5, 5),
activation='relu'))
    model.add(ks.layers.MaxPooling2D(pool_size=(2, 2)))
    print("\nAfter layer2 and maxpooling:")
    print(model.output_shape)
    # Flatten the output from the convolutional layers
    model.add(ks.layers.Flatten())

    # Fully connected layers
    model.add(ks.layers.Dense(120, activation='relu'))
    print("\nAfter layer3:")
    print(model.output_shape)
    model.add(ks.layers.Dense(84, activation='relu'))
    print("\nAfter layer4:")
    print(model.output_shape)
    # Output layer with softmax activation for multi-class
classification
    model.add(ks.layers.Dense(num_classes, activation='softmax'))
    print("\nAfter output layer:")
    print(num_classes)
    return model
build_lenet5((32, 32, 3), 3)
```

```
After layer1 and maxpooling:
(None, 14, 14, 6)

After layer2 and maxpooling:
(None, 5, 5, 16)

After layer3:
```

```
(None, 120)

After layer4:
(None, 84)

After output layer:
3

<Sequential name=sequential, built=True>
```

Task 1.1.2 Train network

Train the network with a

- batch size of 64 samples
- for 20 epochs

```
# Import the f1_score function from utilities
from utilities import f1_score

# Build the LeNet-5 model
input_shape = (32, 32, 3)  # 32x32 RGB images
num_classes = 3  # buildings, forest, sea

model = build_lenet5(input_shape, num_classes)

# Display model summary
model.summary()

# Compile the model
model.compile(
    loss='categorical_crossentropy',
    optimizer='adam',
    metrics=['accuracy', f1_score]
)

# Train the model
batch_size = 64
epochs = 20

print(f"\nTraining the model with batch_size={batch_size} for {epochs}
epochs...")

history = model.fit(
    X_train, y_train,
    batch_size=batch_size,
    epochs=epochs,
    validation_data=(X_val, y_val),
    verbose=1
)
```

```
print("\nTraining completed!")


After layer1 and maxpooling:
(None, 14, 14, 6)

After layer2 and maxpooling:
(None, 5, 5, 16)

After layer3:
(None, 120)

After layer4:
(None, 84)

After output layer:
3

Model: "sequential_1"
```

Layer (type)	Output Shape	Param #
conv2d_2 (Conv2D)	(None, 28, 28, 6)	456
max_pooling2d_2 (MaxPooling2D)	(None, 14, 14, 6)	0
conv2d_3 (Conv2D)	(None, 10, 10, 16)	2,416
max_pooling2d_3 (MaxPooling2D)	(None, 5, 5, 16)	0
flatten_1 (Flatten)	(None, 400)	0
dense_3 (Dense)	(None, 120)	48,120

```
| dense_4 (Dense)                 | (None, 84)             |
10,164 |
├─────────────────────────────────┼────────────────────────┼──────────
──────┤
| dense_5 (Dense)                 | (None, 3)              |
255 |
└─────────────────────────────────┴────────────────────────┴──────────
──────┘

 Total params: 61,411 (239.89 KB)

 Trainable params: 61,411 (239.89 KB)

 Non-trainable params: 0 (0.00 B)


Training the model with batch_size=64 for 20 epochs...
Epoch 1/20
95/95 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.3796 - f1_score:
0.3865 - loss: 1.0771 - val_accuracy: 0.5727 - val_f1_score: 0.9113 -
val_loss: 0.9093
Epoch 2/20
95/95 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.5861 - f1_score:
0.8267 - loss: 0.8504 - val_accuracy: 0.6380 - val_f1_score: 0.8631 -
val_loss: 0.7679
Epoch 3/20
95/95 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.6109 - f1_score:
0.8455 - loss: 0.8172 - val_accuracy: 0.6395 - val_f1_score: 0.8810 -
val_loss: 0.7605
Epoch 4/20
95/95 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.6216 - f1_score:
0.8493 - loss: 0.7977 - val_accuracy: 0.6573 - val_f1_score: 0.8681 -
val_loss: 0.7399
Epoch 5/20
95/95 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.6363 - f1_score:
0.8610 - loss: 0.7838 - val_accuracy: 0.6647 - val_f1_score: 0.8860 -
val_loss: 0.7211
Epoch 6/20
95/95 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.6509 - f1_score:
0.8671 - loss: 0.7640 - val_accuracy: 0.6751 - val_f1_score: 0.8961 -
val_loss: 0.7089
Epoch 7/20
95/95 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.6605 - f1_score:
0.8673 - loss: 0.7644 - val_accuracy: 0.6780 - val_f1_score: 0.8922 -
val_loss: 0.7015
Epoch 8/20
95/95 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.6770 - f1_score:
0.8811 - loss: 0.7381 - val_accuracy: 0.6840 - val_f1_score: 0.8949 -
val_loss: 0.6899
Epoch 9/20
```

```
95/95 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.6869 - f1_score:
0.8869 - loss: 0.7276 - val_accuracy: 0.6929 - val_f1_score: 0.9034 -
val_loss: 0.6790
Epoch 10/20
95/95 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.6844 - f1_score:
0.8812 - loss: 0.7210 - val_accuracy: 0.6795 - val_f1_score: 0.8910 -
val_loss: 0.7001
Epoch 11/20
95/95 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.6902 - f1_score:
0.8820 - loss: 0.7170 - val_accuracy: 0.7003 - val_f1_score: 0.9207 -
val_loss: 0.6756
Epoch 12/20
95/95 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.6971 - f1_score:
0.8879 - loss: 0.7068 - val_accuracy: 0.6899 - val_f1_score: 0.8822 -
val_loss: 0.7161
Epoch 13/20
95/95 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.7011 - f1_score:
0.8861 - loss: 0.7014 - val_accuracy: 0.7047 - val_f1_score: 0.9143 -
val_loss: 0.6584
Epoch 14/20
95/95 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.7095 - f1_score:
0.8887 - loss: 0.6909 - val_accuracy: 0.7151 - val_f1_score: 0.9124 -
val_loss: 0.6615
Epoch 15/20
95/95 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.7098 - f1_score:
0.8852 - loss: 0.6885 - val_accuracy: 0.7270 - val_f1_score: 0.9028 -
val_loss: 0.6489
Epoch 16/20
95/95 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.7126 - f1_score:
0.8904 - loss: 0.6834 - val_accuracy: 0.7300 - val_f1_score: 0.9276 -
val_loss: 0.6416
Epoch 17/20
95/95 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.7143 - f1_score:
0.8885 - loss: 0.6813 - val_accuracy: 0.7359 - val_f1_score: 0.9102 -
val_loss: 0.6385
Epoch 18/20
95/95 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.7230 - f1_score:
0.8901 - loss: 0.6679 - val_accuracy: 0.7418 - val_f1_score: 0.9049 -
val_loss: 0.6370
Epoch 19/20
95/95 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.7242 - f1_score:
0.8882 - loss: 0.6684 - val_accuracy: 0.7448 - val_f1_score: 0.9076 -
val_loss: 0.6394
Epoch 20/20
95/95 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.7280 - f1_score:
0.8908 - loss: 0.6632 - val_accuracy: 0.7389 - val_f1_score: 0.9158 -
val_loss: 0.6248

Training completed!
```

Task 1.1.3

Experiment with different architectures of your choice. Vary the number of filters, try different kernel sizes, add more layers, dropout, early stopping, and modify the fully connected layers. Report the best performance you achieve.

```
#tweaking the model a bit
def build_lenet5(input_shape, num_classes):
    model = ks.Sequential()
    model.add(ks.Input(shape=input_shape))

    # Layer 1: Conv -> ReLU -> MaxPool
    model.add(ks.layers.Conv2D(6, kernel_size=(5, 5), padding='same',
activation='relu'))
    model.add(ks.layers.BatchNormalization())
    model.add(ks.layers.MaxPooling2D(pool_size=(2, 2)))

    # Layer 2: Conv -> ReLU -> MaxPool
    #added in some more dropout to combat overfitting
    model.add(ks.layers.Conv2D(16, kernel_size=(5, 5), padding='same',
activation='relu', kernel_regularizer=ks.regularizers.l2(1e-4)))
    model.add(ks.layers.SpatialDropout2D(0.3))
    model.add(ks.layers.BatchNormalization())
    model.add(ks.layers.MaxPooling2D(pool_size=(2, 2)))

    model.add(ks.layers.Flatten())

    # Fully connected layers
    model.add(ks.layers.Dense(120, activation='relu',
kernel_regularizer=ks.regularizers.l2(1e-4)))
    model.add(ks.layers.Dropout(0.5))

    model.add(ks.layers.Dense(84, activation='relu',
kernel_regularizer=ks.regularizers.l2(1e-4)))
    model.add(ks.layers.Dropout(0.5))

    # Output layer with softmax activation for multi-class
classification
    model.add(ks.layers.Dense(num_classes, activation='softmax'))

    return model

model = build_lenet5((32, 32, 3), 3)

#early stopping and learning rate
callback = ks.callbacks.EarlyStopping(monitor="val_loss", verbose=1,
restore_best_weights=True, patience=15, min_delta=0.0001)
optimizer = ks.optimizers.Adam(learning_rate=1e-3)

# Compile the model
```

```
model.compile(
    loss='categorical_crossentropy',
    optimizer=optimizer,
    metrics=['accuracy', f1_score]
)

# Train the model
batch_size = 64
epochs = 100

print(f"\nTraining the model with batch_size={batch_size} for {epochs}
epochs...")

history = model.fit(
    X_train, y_train,
    batch_size=batch_size,
    epochs=epochs,
    validation_data=(X_val, y_val),
    verbose=1,
    callbacks=[callback]
)

print("\nTraining completed!")
```

```
Training the model with batch_size=64 for 100 epochs...
Epoch 1/100
95/95 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - accuracy: 0.5808 - f1_score:
0.8045 - loss: 0.9307 - val_accuracy: 0.3383 - val_f1_score:
0.0000e+00 - val_loss: 1.1451
Epoch 2/100
95/95 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - accuracy: 0.7440 - f1_score:
0.8604 - loss: 0.6708 - val_accuracy: 0.3383 - val_f1_score:
0.0000e+00 - val_loss: 1.1591
Epoch 3/100
95/95 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - accuracy: 0.8016 - f1_score:
0.8814 - loss: 0.5555 - val_accuracy: 0.3383 - val_f1_score:
0.0000e+00 - val_loss: 1.1663
Epoch 4/100
95/95 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - accuracy: 0.8223 - f1_score:
0.8940 - loss: 0.5133 - val_accuracy: 0.3412 - val_f1_score: 0.1515 -
val_loss: 1.1660
Epoch 5/100
95/95 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - accuracy: 0.8303 - f1_score:
0.8981 - loss: 0.4912 - val_accuracy: 0.5549 - val_f1_score: 0.9306 -
val_loss: 0.9884
Epoch 6/100
95/95 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.8395 - f1_score:
0.9028 - loss: 0.4573 - val_accuracy: 0.6602 - val_f1_score: 0.8968 -
val_loss: 0.7500
```

```
Epoch 7/100
95/95 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - accuracy: 0.8352 - f1_score:
0.8962 - loss: 0.4606 - val_accuracy: 0.7685 - val_f1_score: 0.9469 -
val_loss: 0.5819
Epoch 8/100
95/95 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - accuracy: 0.8479 - f1_score:
0.9055 - loss: 0.4353 - val_accuracy: 0.8546 - val_f1_score: 0.9399 -
val_loss: 0.4383
Epoch 9/100
95/95 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - accuracy: 0.8540 - f1_score:
0.9166 - loss: 0.4257 - val_accuracy: 0.8680 - val_f1_score: 0.9445 -
val_loss: 0.3923
Epoch 10/100
95/95 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - accuracy: 0.8598 - f1_score:
0.9152 - loss: 0.4094 - val_accuracy: 0.8813 - val_f1_score: 0.9334 -
val_loss: 0.3674
Epoch 11/100
95/95 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - accuracy: 0.8618 - f1_score:
0.9179 - loss: 0.4064 - val_accuracy: 0.7849 - val_f1_score: 0.8673 -
val_loss: 0.5484
Epoch 12/100
95/95 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - accuracy: 0.8611 - f1_score:
0.9194 - loss: 0.3973 - val_accuracy: 0.8843 - val_f1_score: 0.9376 -
val_loss: 0.3527
Epoch 13/100
95/95 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - accuracy: 0.8712 - f1_score:
0.9245 - loss: 0.3770 - val_accuracy: 0.8976 - val_f1_score: 0.9546 -
val_loss: 0.3482
Epoch 14/100
95/95 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - accuracy: 0.8814 - f1_score:
0.9307 - loss: 0.3625 - val_accuracy: 0.8858 - val_f1_score: 0.9404 -
val_loss: 0.3293
Epoch 15/100
95/95 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.8779 - f1_score:
0.9363 - loss: 0.3579 - val_accuracy: 0.8828 - val_f1_score: 0.9399 -
val_loss: 0.3553
Epoch 16/100
95/95 ━━━━━━━━━━━━━━━━━━━━ 1s 9ms/step - accuracy: 0.8850 - f1_score:
0.9359 - loss: 0.3420 - val_accuracy: 0.8872 - val_f1_score: 0.9366 -
val_loss: 0.3588
Epoch 17/100
95/95 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.8852 - f1_score:
0.9293 - loss: 0.3462 - val_accuracy: 0.8872 - val_f1_score: 0.9338 -
val_loss: 0.3449
Epoch 18/100
95/95 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.8905 - f1_score:
0.9413 - loss: 0.3386 - val_accuracy: 0.8947 - val_f1_score: 0.9416 -
val_loss: 0.3374
Epoch 19/100
```

```
95/95 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.8938 - f1_score:
0.9380 - loss: 0.3316 - val_accuracy: 0.8932 - val_f1_score: 0.9364 -
val_loss: 0.3580
Epoch 20/100
95/95 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.8977 - f1_score:
0.9444 - loss: 0.3270 - val_accuracy: 0.8635 - val_f1_score: 0.8951 -
val_loss: 0.4164
Epoch 21/100
95/95 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.8972 - f1_score:
0.9449 - loss: 0.3205 - val_accuracy: 0.8813 - val_f1_score: 0.9258 -
val_loss: 0.3589
Epoch 22/100
95/95 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.9018 - f1_score:
0.9462 - loss: 0.3077 - val_accuracy: 0.8620 - val_f1_score: 0.8928 -
val_loss: 0.4131
Epoch 23/100
95/95 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.9037 - f1_score:
0.9474 - loss: 0.3027 - val_accuracy: 0.8442 - val_f1_score: 0.8950 -
val_loss: 0.4629
Epoch 24/100
95/95 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.9124 - f1_score:
0.9502 - loss: 0.2855 - val_accuracy: 0.8665 - val_f1_score: 0.9178 -
val_loss: 0.3850
Epoch 25/100
95/95 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.9139 - f1_score:
0.9493 - loss: 0.2911 - val_accuracy: 0.8902 - val_f1_score: 0.9552 -
val_loss: 0.3501
Epoch 26/100
95/95 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.9045 - f1_score:
0.9455 - loss: 0.2955 - val_accuracy: 0.8442 - val_f1_score: 0.8942 -
val_loss: 0.4515
Epoch 27/100
95/95 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.9142 - f1_score:
0.9511 - loss: 0.2803 - val_accuracy: 0.8902 - val_f1_score: 0.9426 -
val_loss: 0.3498
Epoch 28/100
95/95 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.9144 - f1_score:
0.9477 - loss: 0.2793 - val_accuracy: 0.8531 - val_f1_score: 0.9163 -
val_loss: 0.4681
Epoch 29/100
95/95 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.9175 - f1_score:
0.9575 - loss: 0.2680 - val_accuracy: 0.8858 - val_f1_score: 0.9436 -
val_loss: 0.3709
Epoch 29: early stopping
Restoring model weights from the end of the best epoch: 14.

Training completed!
```

Task 1.2 Evaluaiton

Task 1.2.1 Plot training history

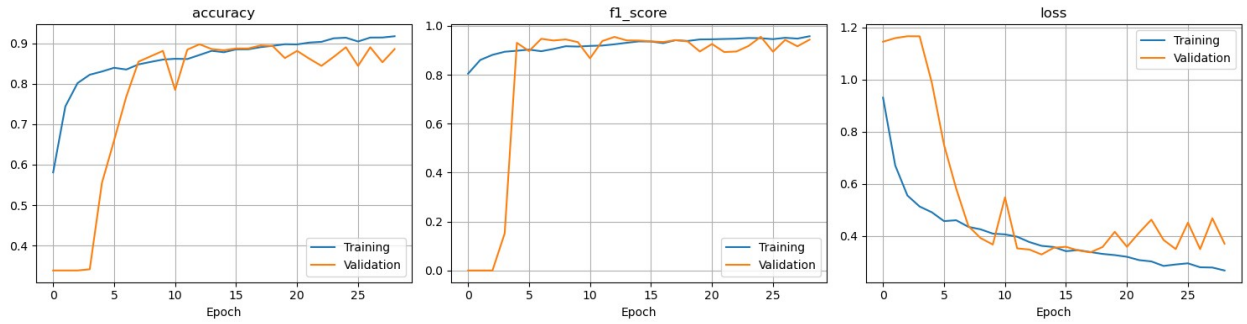
- Plot the training/validation accuracy and loss curves (plot_training_history() is in utilities.py).
- Report the final validation accuracy (f1_score() is in utilities.py).

```
# Plot training history
from utilities import plot_training_history

# Plot the training history
plot_training_history(history)

# Report final validation accuracy and F1 score
final_val_accuracy = history.history['val_accuracy'][-1]
final_val_f1 = history.history['val_f1_score'][-1]

print(f"\nFinal Validation Results:")
print(f"Accuracy: {final_val_accuracy:.4f}")
print(f"F1 Score: {final_val_f1:.4f}")
#tf.keras.utils.plot_model(model, show_shapes=True,
show_layer_activations=True, show_layer_names=True, dpi=100) #plotting
model architecture
```




```
Final Validation Results:
Accuracy: 0.8858
F1 Score: 0.9436
```

Task 1.2.2 Evaluate on the test dataset

- Report test accuracy (and F1 score (f1_score() is in utilities.py)).

```
# Evaluate on test dataset
test_loss, test_accuracy, test_f1 = model.evaluate(X_test, y_test,
verbose=0)

print("Test Results:")
print(f"Test Loss: {test_loss:.4f}")
```

```
print(f"Test Accuracy: {test_accuracy:.4f}")
print(f"Test F1 Score: {test_f1:.4f}")

# Get predictions for confusion matrix
y_pred = model.predict(X_test)
y_pred_classes = np.argmax(y_pred, axis=1)
y_true_classes = np.argmax(y_test, axis=1)

print(f"\nTest predictions shape: {y_pred.shape}")
print(f"Predicted classes shape: {y_pred_classes.shape}")
print(f"True classes shape: {y_true_classes.shape}")
```

```
Test Results:
Test Loss: 0.3780
Test Accuracy: 0.8557
Test F1 Score: 0.3385
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step

Test predictions shape: (1421, 3)
Predicted classes shape: (1421,)
True classes shape: (1421,)
```

Task 1.2.3 Create a confusion matrix for both training and testing data

- Visualize confusion matrices.
- Do the test data and train data predict the same items wrong?

```
from sklearn.metrics import confusion_matrix
import seaborn as sns

# Create confusion matrices
def plot_confusion_matrix(y_true, y_pred, classes, title):
    """Plot confusion matrix."""
    cm = confusion_matrix(y_true, y_pred)
    plt.figure(figsize=(8, 6))
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
                xticklabels=classes, yticklabels=classes)
    plt.title(title)
    plt.xlabel('Predicted')
    plt.ylabel('Actual')
    plt.show()
    return cm

# Get training predictions
y_train_pred = model.predict(X_train)
y_train_pred_classes = np.argmax(y_train_pred, axis=1)
y_train_true_classes = np.argmax(y_train, axis=1)

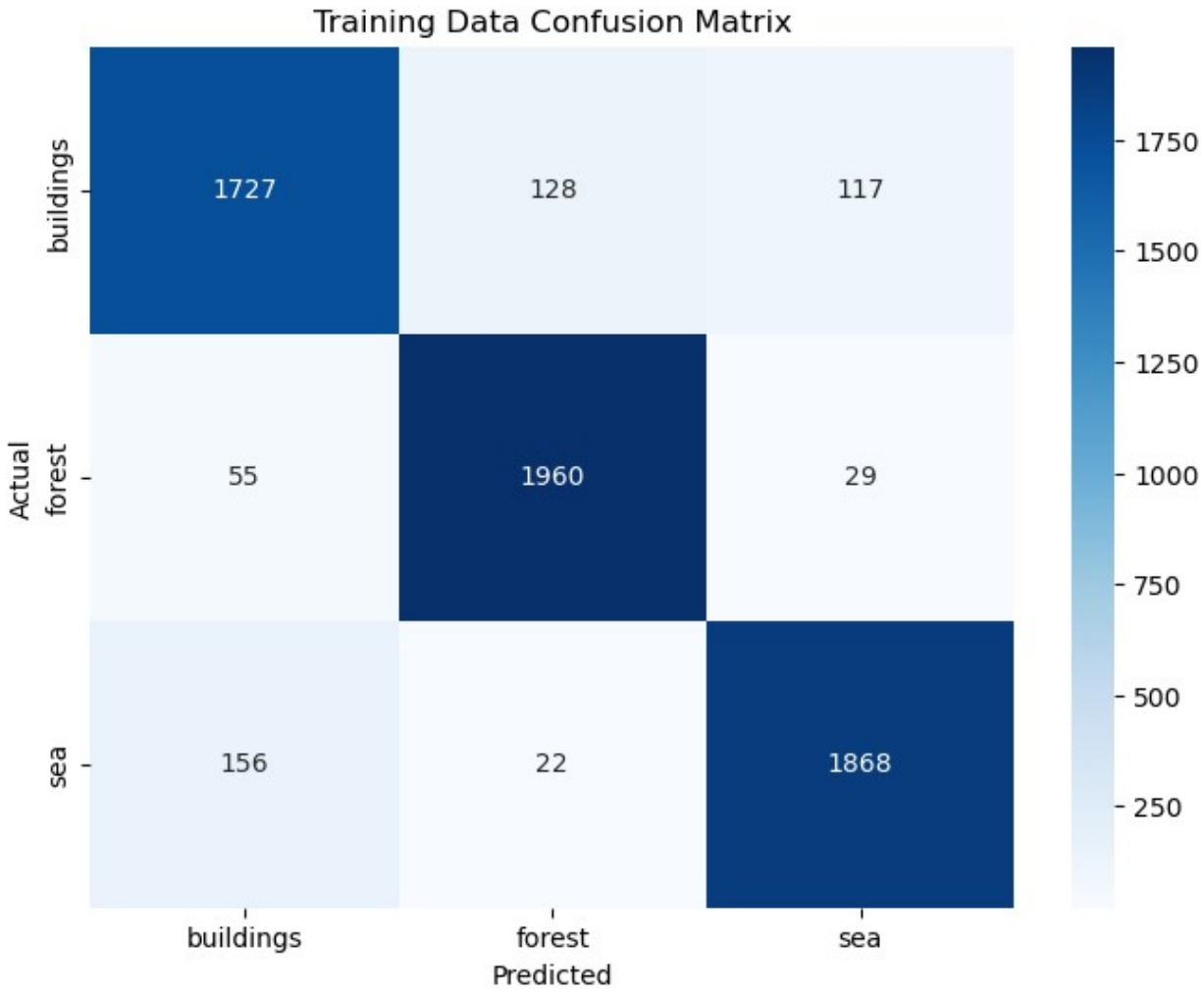
# Plot confusion matrices
print("Confusion Matrix - Training Data:")
train_cm = plot_confusion_matrix(y_train_true_classes,
```

```
y_train_pred_classes,
                                    class_names, "Training Data Confusion
Matrix")

print("\nConfusion Matrix - Test Data:")
test_cm = plot_confusion_matrix(y_true_classes, y_pred_classes,
                                   class_names, "Test Data Confusion
Matrix")

# the model seems overly biased towards buildings, this could make
sense as buildings often have more diverse looks than forests and the
sea
```

```
190/190 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step
Confusion Matrix - Training Data:
```




```
Confusion Matrix - Test Data:
```



Part 2: Robustness (choose two or more), You may use add_gaussian_noise() and add_motion_blur() from utilities.py

Task 2.1 Data Augmentation (training-time)

- Implement an augmentation pipeline.
- Train a model with augmentation and compare against the baseline from Part 1.

Task 2.2 Noise Robustness (test-time corruptions)

- Create corrupted versions of the test images and evaluate the baseline model:
  - Gaussian noise
  - Motion blur
- Report accuracy on the clean test set vs. each corrupted test set.

Task 2.3 Discussion (no coding)

- Which noise types affect the model most?
- What techniques could improve robustness (data augmentation, adversarial training, denoising pre-processing, larger models)?

```
from utilities import add_gaussian_noise, add_motion_blur

X_test_noisy = add_gaussian_noise(X_test)
X_test_blur = add_motion_blur(X_test)

# Clean test accuracy
clean_acc = model.evaluate(X_test, y_test, verbose=0)[1]

# Gaussian noise
x_test_noisy = add_gaussian_noise(X_test)
noisy_acc = model.evaluate(X_test_noisy, y_test, verbose=0)[1]

# Motion blur
x_test_blur = add_motion_blur(X_test)
blur_acc = model.evaluate(X_test_blur, y_test, verbose=0)[1]

print(f"Clean test accuracy:     {clean_acc:.4f}")
print(f"Gaussian noise accuracy: {noisy_acc:.4f}")
print(f"Motion blur accuracy:    {blur_acc:.4f}")
```

```
Clean test accuracy:     0.8557
Gaussian noise accuracy: 0.3075
Motion blur accuracy:    0.5032
```

```
#sourced from
https://www.tensorflow.org/tutorials/images/data_augmentation
#augmentation pipeline

from tensorflow.keras import layers

data_augmentation = tf.keras.Sequential([
    layers.RandomFlip("horizontal_and_vertical"),
    layers.RandomRotation(0.2),
    layers.RandomContrast(factor=0.2)
])

#tweaking the model a bit
def build_lenet5(input_shape, num_classes):
    model = ks.Sequential()
    model.add(ks.Input(shape=input_shape))
    model.add(data_augmentation)

    # Layer 1: Conv -> ReLU -> MaxPool
    model.add(ks.layers.Conv2D(6, kernel_size=(5, 5), padding='same',
activation='relu'))
    model.add(ks.layers.BatchNormalization())
```

```
    model.add(ks.layers.MaxPooling2D(pool_size=(2, 2)))

    # Layer 2: Conv -> ReLU -> MaxPool
    #added in some more dropout to combat overfitting
    model.add(ks.layers.Conv2D(16, kernel_size=(5, 5), padding='same',
activation='relu', kernel_regularizer=ks.regularizers.l2(1e-4)))
    model.add(ks.layers.SpatialDropout2D(0.3))
    model.add(ks.layers.BatchNormalization())
    model.add(ks.layers.MaxPooling2D(pool_size=(2, 2)))

    model.add(ks.layers.Flatten())

    # Fully connected layers
    model.add(ks.layers.Dense(120, activation='relu',
kernel_regularizer=ks.regularizers.l2(1e-4)))
    model.add(ks.layers.Dropout(0.5))

    model.add(ks.layers.Dense(84, activation='relu',
kernel_regularizer=ks.regularizers.l2(1e-4)))
    model.add(ks.layers.Dropout(0.5))

    # Output layer with softmax activation for multi-class
classification
    model.add(ks.layers.Dense(num_classes, activation='softmax'))

    return model

model = build_lenet5((32, 32, 3), 3)

#early stopping and learning rate
callback = ks.callbacks.EarlyStopping(monitor="val_loss", verbose=1,
restore_best_weights=True, patience=15, min_delta=0.0001)
optimizer = ks.optimizers.Adam(learning_rate=1e-3)

# Compile the model
model.compile(
    loss='categorical_crossentropy',
    optimizer=optimizer,
    metrics=['accuracy', f1_score]
)

# Train the model
batch_size = 64
epochs = 100

print(f"\nTraining the model with batch_size={batch_size} for {epochs}
epochs...")

history = model.fit(
    X_train, y_train,
```

```
    batch_size=batch_size,
    epochs=epochs,
    validation_data=(X_val, y_val),
    verbose=1,
    callbacks=[callback]
)

print("\nTraining completed!")
```

```
Training the model with batch_size=64 for 100 epochs...
Epoch 1/100
95/95 ━━━━━━━━━━━━━━━━━━━━ 2s 11ms/step - accuracy: 0.6031 - f1_score:
0.8095 - loss: 0.8854 - val_accuracy: 0.3383 - val_f1_score:
0.0000e+00 - val_loss: 1.1320
Epoch 2/100
95/95 ━━━━━━━━━━━━━━━━━━━━ 1s 9ms/step - accuracy: 0.6843 - f1_score:
0.8521 - loss: 0.7486 - val_accuracy: 0.3961 - val_f1_score: 0.9091 -
val_loss: 1.1219
Epoch 3/100
95/95 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.7179 - f1_score:
0.8664 - loss: 0.7123 - val_accuracy: 0.3383 - val_f1_score:
0.0000e+00 - val_loss: 1.1650
Epoch 4/100
95/95 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.7364 - f1_score:
0.8676 - loss: 0.6739 - val_accuracy: 0.5163 - val_f1_score: 0.9802 -
val_loss: 1.0446
Epoch 5/100
95/95 ━━━━━━━━━━━━━━━━━━━━ 1s 9ms/step - accuracy: 0.7456 - f1_score:
0.8826 - loss: 0.6563 - val_accuracy: 0.5178 - val_f1_score: 0.9807 -
val_loss: 1.0153
Epoch 6/100
95/95 ━━━━━━━━━━━━━━━━━━━━ 1s 9ms/step - accuracy: 0.7527 - f1_score:
0.8825 - loss: 0.6399 - val_accuracy: 0.5119 - val_f1_score: 0.7278 -
val_loss: 0.9863
Epoch 7/100
95/95 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.7592 - f1_score:
0.8830 - loss: 0.6242 - val_accuracy: 0.7433 - val_f1_score: 0.9092 -
val_loss: 0.5776
Epoch 8/100
95/95 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.7662 - f1_score:
0.8876 - loss: 0.6154 - val_accuracy: 0.3487 - val_f1_score: 0.1267 -
val_loss: 2.2287
Epoch 9/100
95/95 ━━━━━━━━━━━━━━━━━━━━ 1s 9ms/step - accuracy: 0.7710 - f1_score:
0.8897 - loss: 0.6002 - val_accuracy: 0.4303 - val_f1_score: 0.4270 -
val_loss: 1.1638
Epoch 10/100
95/95 ━━━━━━━━━━━━━━━━━━━━ 1s 9ms/step - accuracy: 0.7674 - f1_score:
0.8947 - loss: 0.6017 - val_accuracy: 0.7151 - val_f1_score: 0.9031 -
```

```
val_loss: 0.6294
Epoch 11/100
95/95 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.7788 - f1_score:
0.8934 - loss: 0.5971 - val_accuracy: 0.5608 - val_f1_score: 0.8611 -
val_loss: 1.0721
Epoch 12/100
95/95 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.7742 - f1_score:
0.8976 - loss: 0.5929 - val_accuracy: 0.5905 - val_f1_score: 0.8373 -
val_loss: 0.9769
Epoch 13/100
95/95 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.7841 - f1_score:
0.9019 - loss: 0.5758 - val_accuracy: 0.7478 - val_f1_score: 0.9208 -
val_loss: 0.6584
Epoch 14/100
95/95 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.7816 - f1_score:
0.8988 - loss: 0.5704 - val_accuracy: 0.8665 - val_f1_score: 0.9540 -
val_loss: 0.3836
Epoch 15/100
95/95 ━━━━━━━━━━━━━━━━━━━━ 1s 9ms/step - accuracy: 0.7921 - f1_score:
0.9034 - loss: 0.5635 - val_accuracy: 0.8056 - val_f1_score: 0.9007 -
val_loss: 0.4852
Epoch 16/100
95/95 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.7829 - f1_score:
0.8973 - loss: 0.5648 - val_accuracy: 0.8116 - val_f1_score: 0.9235 -
val_loss: 0.4717
Epoch 17/100
95/95 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.8009 - f1_score:
0.9057 - loss: 0.5471 - val_accuracy: 0.7003 - val_f1_score: 0.8387 -
val_loss: 0.7585
Epoch 18/100
95/95 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.7915 - f1_score:
0.9011 - loss: 0.5546 - val_accuracy: 0.7685 - val_f1_score: 0.8597 -
val_loss: 0.5909
Epoch 19/100
95/95 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.7936 - f1_score:
0.9068 - loss: 0.5575 - val_accuracy: 0.8724 - val_f1_score: 0.9406 -
val_loss: 0.3955
Epoch 20/100
95/95 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.7920 - f1_score:
0.9024 - loss: 0.5683 - val_accuracy: 0.8635 - val_f1_score: 0.9385 -
val_loss: 0.3986
Epoch 21/100
95/95 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.7888 - f1_score:
0.9000 - loss: 0.5574 - val_accuracy: 0.8220 - val_f1_score: 0.8899 -
val_loss: 0.4528
Epoch 22/100
95/95 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.7936 - f1_score:
0.9024 - loss: 0.5513 - val_accuracy: 0.5861 - val_f1_score: 0.8954 -
val_loss: 1.1965
```

```
Epoch 23/100
95/95 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.8001 - f1_score:
0.9053 - loss: 0.5363 - val_accuracy: 0.5757 - val_f1_score: 0.8785 -
val_loss: 1.0714
Epoch 24/100
95/95 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.7897 - f1_score:
0.9059 - loss: 0.5533 - val_accuracy: 0.8205 - val_f1_score: 0.9081 -
val_loss: 0.5231
Epoch 25/100
95/95 ━━━━━━━━━━━━━━━━━━━━ 1s 9ms/step - accuracy: 0.7953 - f1_score:
0.9080 - loss: 0.5560 - val_accuracy: 0.6691 - val_f1_score: 0.8582 -
val_loss: 0.7975
Epoch 26/100
95/95 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.8012 - f1_score:
0.9067 - loss: 0.5409 - val_accuracy: 0.6795 - val_f1_score: 0.9150 -
val_loss: 0.7909
Epoch 27/100
95/95 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.7946 - f1_score:
0.9004 - loss: 0.5404 - val_accuracy: 0.8531 - val_f1_score: 0.9320 -
val_loss: 0.4219
Epoch 28/100
95/95 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.8034 - f1_score:
0.9060 - loss: 0.5315 - val_accuracy: 0.7626 - val_f1_score: 0.9506 -
val_loss: 0.5894
Epoch 29/100
95/95 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.8040 - f1_score:
0.9112 - loss: 0.5422 - val_accuracy: 0.8309 - val_f1_score: 0.8726 -
val_loss: 0.5000
Epoch 29: early stopping
Restoring model weights from the end of the best epoch: 14.

Training completed!
```

2.3

According to the given predicted data, the Gaussian Noise model affected the model the most.

This can be adjusted by simply denoising the data and augment this to improve the accuracy of the corrupted model.

```python
test_loss_noisy, test_accuracy_noisy, test_f1_noisy =
model.evaluate(X_test_noisy, y_test, verbose=0)

print("Test Results (Noisy):")
print(f"Test Loss: {test_loss_noisy:.4f}")
print(f"Test Accuracy: {test_accuracy_noisy:.4f}")
print(f"Test F1 Score: {test_f1_noisy:.4f}")

#predictions for noisy data
y_pred_noisy = model.predict(X_test_noisy)
```

```python
y_pred_classes_noisy = np.argmax(y_pred_noisy, axis=1)
y_true_classes_noisy = np.argmax(y_test, axis=1)

print("\nConfusion Matrix - Test Data (noisy):")
test_cm_noisy = plot_confusion_matrix(
    y_true_classes_noisy,
    y_pred_classes_noisy,
    class_names,
    "Test Data Confusion Matrix (noisy)"
)

test_loss_blurry, test_accuracy_blurry, test_f1_blurry =
model.evaluate(X_test_blur, y_test, verbose=0)

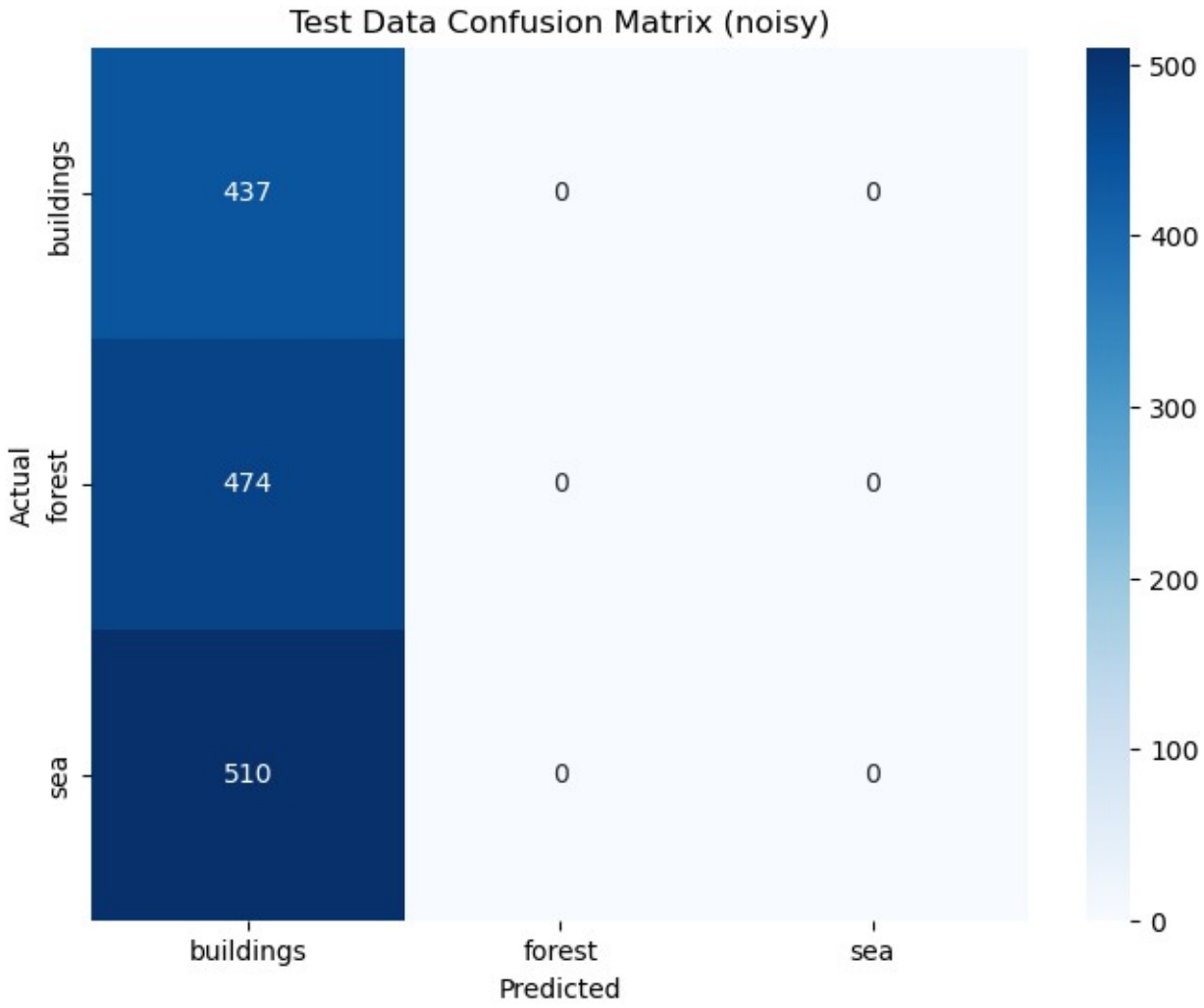
print("\nTest Results (Blurry):")
print(f"Test Loss: {test_loss_blurry:.4f}")
print(f"Test Accuracy: {test_accuracy_blurry:.4f}")
print(f"Test F1 Score: {test_f1_blurry:.4f}")

#predictions for blurry data
y_pred_blurry = model.predict(X_test_blur)
y_pred_classes_blurry = np.argmax(y_pred_blurry, axis=1)
y_true_classes_blurry = np.argmax(y_test, axis=1)

print("\nConfusion Matrix - Test Data (blurry):")
test_cm_blurry = plot_confusion_matrix(
    y_true_classes_blurry,
    y_pred_classes_blurry,
    class_names,
    "Test Data Confusion Matrix (blurry)"
)
```

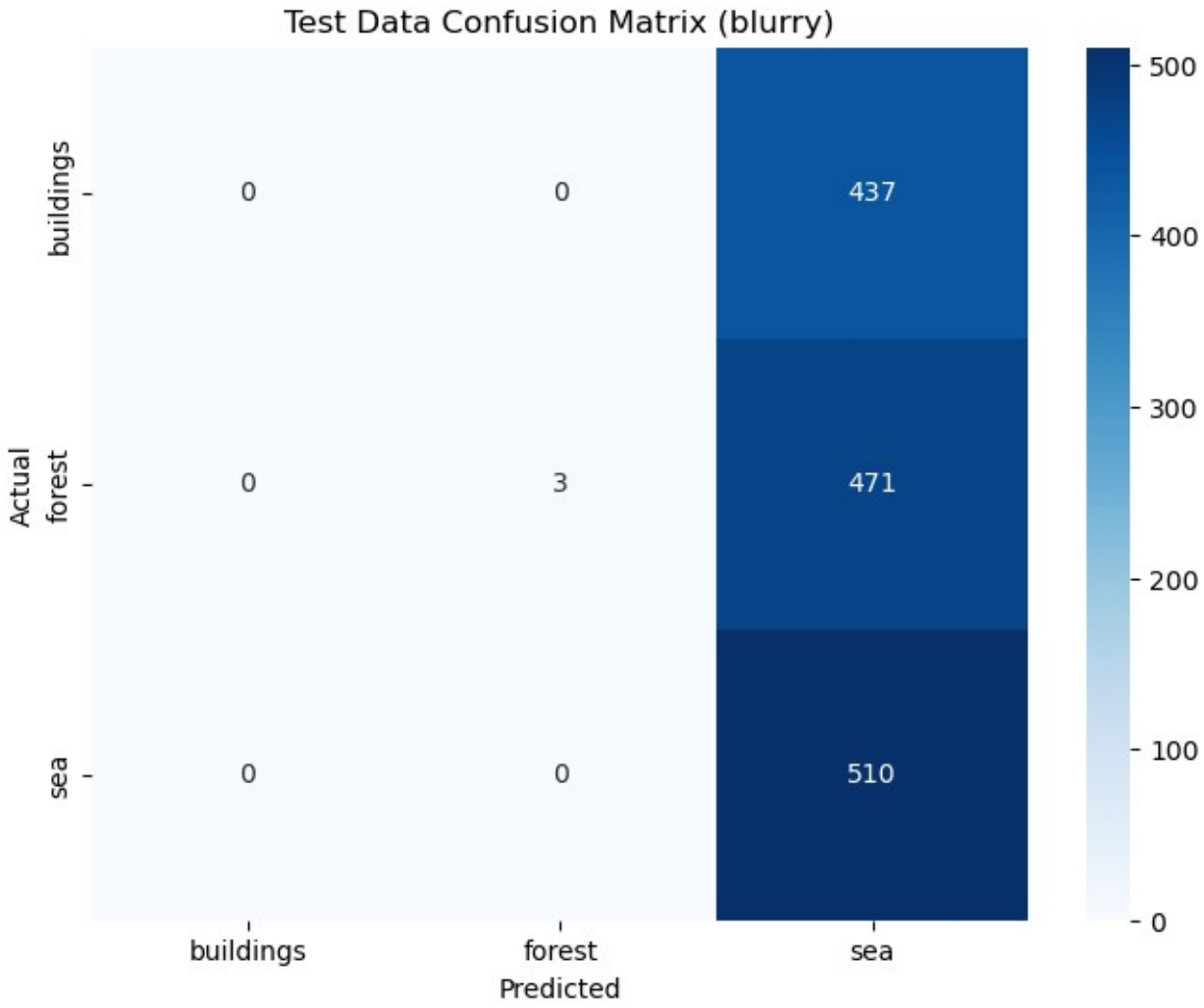
```
Test Results (Noisy):
Test Loss: 228.4646
Test Accuracy: 0.3075
Test F1 Score: 0.0000
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step

Confusion Matrix - Test Data (noisy):
```




```
Test Results (Blurry):
Test Loss: 2.6960
Test Accuracy: 0.3610
Test F1 Score: 0.0444
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step

Confusion Matrix - Test Data (blurry):
```



Part 3: Results & Comparison

- Summarize results (table/plot):
  - Baseline (clean) vs. Augmented (if attempted)
  - Clean vs. each corrupted test set (Task 2.2)
- Brief discussion:
  - Which corruptions hurt most, and why?
  - Did augmentation help? Which transforms mattered?
  - What would you try next (e.g., stronger augmentation, adversarial training, denoising pre-processing, larger models, early stopping, ensembling)?

The "clean" model managed to perform better than the corrupted models bu quite a large margin.

The Gaussian Corruption hurt the accuracy the most. This may be caused by several things such as random pixel-level fluctations to the images. The CNN relies on fine-grained local features in

the images, and the Gaussian Noise makes it harder to analyze and predict. Although, it may be beneficial for the model to be able to predict images.

One interesting effect was that blur made the network think almost every image was sea, while noise made it think every image was of a building.

By augmenting the data it allows exposure to the data to improve on feature generalization and regularization. This may reduce overfitting as well as learn higher-level representations. Even with corruption, augmentation allows it to predict patterns with both noise and blur. This is ideal considering that images and data not always are "clean". Denoising can also be considered, but not too much as it is an advantage for the model to predict with some noise.